

Python Interview Reference

[Decorators](#) · [Dunder Methods](#) · [Generators](#) · [Context Managers](#) · [asyncio](#)

Quick-recall examples for technical interviews

1. Decorators

A decorator wraps a function (or class) to extend behavior without modifying it. It is syntactic sugar for `func = decorator(func)`.

1-A Function Decorator

► Pattern: wrapper function returns inner function

```
import functools

def timer(func):
    # decorator factory
    @functools.wraps(func)
    # preserve metadata
    def wrapper(*args, **kwargs):
        import time
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        print(f'{func.__name__} took {time.perf_counter()-t0:.4f}s')
        return result
    return wrapper

@timer
def slow_add(a, b):
    import time; time.sleep(0.1)
    return a + b
```

```
slow_add(1, 2)  # prints: slow_add took 0.100xs
```

💡 *@functools.wraps copies `__name__`, `__doc__` to wrapper so introspection still works.*

Decorator with arguments (factory pattern)

```
def repeat(n):
    # outer: receives args
    def decorator(func):
        # middle: receives func
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(n):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator
# return the real decorator

@repeat(3)
def greet(name):
    print(f'Hello {name}')
```

```
greet('Li')    # prints Hello Li   three times
```

1-B Class-Based Decorator

► Implement `__call__` — the instance becomes the wrapper

```
class Retry:
    def __init__(self, times=3):
        self.times = times

    def __call__(self, func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, self.times + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f'Attempt {attempt} failed: {e}')
            raise RuntimeError('All retries exhausted')
        return wrapper

@Retry(times=3)
def flaky():
    import random
    if random.random() < 0.7: raise ValueError('oops')
    return 'ok'
```

💡 Class decorators are useful when you need to store state across calls (e.g. call count, cache).

Concept	One-liner
@decorator	func = decorator(func)
@functools.wraps	preserve <code>__name__</code> , <code>__doc__</code>
Decorator with args	3-layer nesting: <code>outer(args) → decorator(func) → wrapper</code>
Class decorator	implement <code>__call__(self, func)</code>
Stacking decorators	<code>@A @B</code> $\equiv$ <code>func = A(B(func))</code> — B applied first

2. Class Dunder (Magic) Methods

Dunder methods let your objects hook into Python's built-in protocols (arithmetic, comparison, containers, iteration, context managers, etc.).

Core Dunders — Quick Reference

Dunder	Triggered by
<code>__init__</code>	<code>MyClass()</code> — initializer
<code>__repr__</code>	<code>repr(obj)</code> , interactive shell
<code>__str__</code>	<code>str(obj)</code> , <code>print()</code>
<code>__len__</code>	<code>len(obj)</code>
<code>__getitem__</code>	<code>obj[key]</code>
<code>__setitem__</code>	<code>obj[key] = val</code>
<code>__contains__</code>	<code>x in obj</code>
<code>__iter__</code>	<code>for x in obj</code> , <code>iter(obj)</code>
<code>__next__</code>	<code>next(obj)</code> — iterator protocol
<code>__enter__</code> / <code>__exit__</code>	<code>with obj:</code>
<code>__eq__</code> / <code>__lt__</code>	<code>==</code> , <code><</code> (enable sorting)
<code>__add__</code>	<code>obj + other</code>
<code>__call__</code>	<code>obj()</code> — callable object

Example: a simple Vector class

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f'Vector({self.x}, {self.y})'

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

    def __abs__(self):
        # abs(v)
```

```

        return (self.x**2 + self.y**2) ** 0.5

    def __len__(self):          # len(v) → int
        return 2

    def __bool__(self):         # bool(v)
        return bool(abs(self))

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)    # Vector(4, 6)
print(abs(v1))    # 2.236...

```

Example: custom container with `__getitem__`

```

class RingBuffer:
    def __init__(self, capacity):
        self._buf = [None] * capacity
        self._cap = capacity
        self._head = 0

    def append(self, val):
        self._buf[self._head % self._cap] = val
        self._head += 1

    def __getitem__(self, idx):      # buf[i]
        return self._buf[idx % self._cap]

    def __len__(self):              # len(buf)
        return min(self._head, self._cap)

    def __repr__(self):
        return f'RingBuffer({self._buf})'

```

💡 If you define `__eq__`, Python sets `__hash__ = None`. Add `__hash__` explicitly for hashable objects.

3. Generators

A generator is a function that yields values lazily. Each call to `next()` resumes after the last yield. State is preserved automatically.

3-A Generator Function

```
def countdown(n):
    while n > 0:
        yield n          # suspend here, return n to caller
        n -= 1           # resumes here on next()

for x in countdown(3): # 3 2 1
    print(x)

# Manual iteration
gen = countdown(2)
print(next(gen))      # 2
print(next(gen))      # 1
# next(gen) → StopIteration
```

3-B Infinite Generator

```
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

import itertools
first10 = list(itertools.islice(fib(), 10))
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

3-C Generator Expression

► Like list comprehension but lazy — no brackets

```
squares = (x*x for x in range(1_000_000)) # O(1) memory

total = sum(x*x for x in range(1000))      # pipeline

# vs list comprehension (eager, loads all into memory)
squares_list = [x*x for x in range(1_000_000)] # O(n) memory
```

3-D send() and yield as expression (coroutine-style)

```
def accumulator():
    total = 0
    while True:
        value = yield total    # yield sends total OUT; receives value IN
        if value is None:
            break
        total += value

acc = accumulator()
next(acc)           # prime: advance to first yield
print(acc.send(10)) # 10
print(acc.send(5))  # 15
print(acc.send(3))  # 18
```

Feature	Notes
yield	pause function, return value to caller
next(gen)	resume from last yield; StopIteration when done
gen.send(val)	resume AND pass val in as yield expression result
gen.throw(exc)	raise exception inside generator at yield point
gen.close()	throw GeneratorExit to cleanly shut down
Generator expr	(x for x in ...) — lazy, O(1) memory
yield from sub()	delegate to sub-generator; passes send/throw through

💡 Generators implement `__iter__` and `__next__` automatically — they ARE iterators.

4. Context Managers

Context managers guarantee setup and teardown via the `with` statement. Python calls `__enter__` on entry and `__exit__` on exit (even on exception).

4-A Class-Based Context Manager

```
class ManagedFile:
    def __init__(self, path, mode='r'):
        self.path, self.mode = path, mode

    def __enter__(self):
        self.file = open(self.path, self.mode)
        return self.file          # bound to as target

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        return False              # False = don't suppress exceptions
```

```
with ManagedFile('data.txt') as f:
    content = f.read()
# file is closed here, even if an exception was raised
```

💡 Return `True` from `__exit__` to suppress the exception. Usually return `False`.

4-B Generator-Based with `@contextmanager`

► Everything before `yield = __enter__`; after `yield = __exit__`

```
from contextlib import contextmanager

@contextmanager
def timer_ctx(label):
    import time
    t0 = time.perf_counter()          # __enter__ logic
    try:
        yield                          # 'as' target = None here
    finally:
        # __exit__ logic
        elapsed = time.perf_counter() - t0
        print(f'{label}: {elapsed:.4f}s')

with timer_ctx('matrix multiply'):
    result = [i*i for i in range(10_000)]
# prints: matrix multiply: 0.0012s
```


4-C Nested & Multiple Context Managers

```
# Python 3.10+ parenthesised form
with (
    open('input.txt') as src,
    open('output.txt', 'w') as dst,
):
    dst.write(src.read().upper())

# Classic (any version)
with open('a') as a, open('b') as b:
    pass
```

4-D contextlib.suppress — swallow specific exceptions

```
from contextlib import suppress

with suppress(FileNotFoundError):
    os.remove('maybe_exists.tmp')    # no try/except needed
```

Method / decorator	Purpose
<code>__enter__(self)</code>	setup; return value bound to <code>as</code> variable
<code>__exit__(exc_t,v,tb)</code>	teardown; return <code>True</code> to suppress exception
<code>@contextmanager</code>	turn generator into context manager (yield once)
<code>contextlib.suppress</code>	context manager that swallows given exception types
<code>contextlib.ExitStack</code>	dynamically compose N context managers

5. asyncio

asyncio enables concurrent I/O-bound code in a single thread using cooperative multitasking. Key primitives: coroutines (async def), Tasks, and the event loop.

5-A Coroutine basics

```
import asyncio

async def fetch(url):          # coroutine function
    print(f'fetching {url}')
    await asyncio.sleep(1)      # yield control back to event loop
    return f'data from {url}'

async def main():
    result = await fetch('api/v1') # await suspends until done
    print(result)

asyncio.run(main()) # entry point – creates & runs event loop
```

💡 *await can only appear inside async def. asyncio.sleep() never blocks the thread.*

5-B Concurrent tasks with asyncio.gather

► **gather runs coroutines concurrently; total time ≈ max(individual times)**

```
async def fetch(url, delay):
    await asyncio.sleep(delay)
    return f'{url} done'

async def main():
    # All three run concurrently – total ~1s, not 1+2+3=6s
    results = await asyncio.gather(
        fetch('api/users', delay=1),
        fetch('api/orders', delay=2),
        fetch('api/products', delay=3),
    )
    for r in results:
        print(r)

asyncio.run(main())
```

5-C `asyncio.create_task` — fire and don't block

```
async def background_job(name):
    await asyncio.sleep(2)
    print(f'{name} done')

async def main():
    task = asyncio.create_task(background_job('sync-db')) # starts now
    print('doing other work...')
    await asyncio.sleep(0.5) # yields; task runs in background
    print('still doing work...')
    await task # wait for task to finish

asyncio.run(main())
# Output: doing other work... → still doing work... → sync-db done
```

5-D `async for` / `async with`

```
# async with – context manager that can await
async def download(url):
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as resp:
            return await resp.text()

# async for – async generator iteration
async def stream_lines(path):
    async with aiofiles.open(path) as f:
        async for line in f:
            yield line.strip()

async def main():
    async for line in stream_lines('big.log'):
        process(line)
```

5-E `asyncio.Queue` — producer / consumer

```
async def producer(queue, items):
    for item in items:
        await queue.put(item)
        await asyncio.sleep(0.1) # simulate I/O
    await queue.put(None) # sentinel

async def consumer(queue):
    while True:
```

```

        item = await queue.get()
        if item is None:
            break
        print(f'processed: {item}')
        queue.task_done()

async def main():
    q = asyncio.Queue(maxsize=5)
    await asyncio.gather(
        producer(q, range(10)),
        consumer(q),
    )

asyncio.run(main())

```

Concept	One-liner
<code>async def f()</code>	defines a coroutine; calling it returns a coroutine object
<code>await expr</code>	suspend current coroutine until <code>expr</code> is complete
<code>asyncio.run(coro)</code>	create event loop, run <code>coro</code> , close loop
<code>asyncio.gather(*cos)</code>	run coroutines concurrently, return list of results
<code>asyncio.create_task</code>	schedule coroutine without awaiting immediately
<code>async with</code>	context manager whose <code>__aenter__</code> / <code>__aexit__</code> are async
<code>async for</code>	iterate over async generator (implements <code>__aiter__</code>)
<code>asyncio.Queue</code>	thread-safe async queue for producer/consumer patterns
<code>asyncio.sleep(0)</code>	yield control to event loop without sleeping (cooperative yield)

💡 *asyncio is single-threaded. Use `concurrent.futures.ThreadPoolExecutor` for CPU-bound work inside async code: `await loop.run_in_executor(executor, blocking_fn)`.*

Quick-Recall Cheat Sheet

Topic	Key Mental Model
Function decorator	func = wrapper(func) — 3-layer if it takes args
Class decorator	implement <code>__call__(self, func)</code> to make it wrappable
<code>@functools.wraps</code>	always use it to preserve metadata
<code>__repr__</code> vs <code>__str__</code>	repr = unambiguous (dev), str = readable (user)
<code>__enter__</code> / <code>__exit__</code>	setup/teardown; <code>__exit__</code> gets exc info; True = suppress
yield in generator	suspends, returns value; <code>next()</code> resumes
<code>send(val)</code> to generator	resume AND inject value; prime with <code>next()</code> first
<code>@contextmanager</code>	yield once: before= <code>enter</code> , after(<code>finally</code>)= <code>exit</code>
<code>asyncio.gather</code>	concurrent tasks; total time $\approx$ slowest, not sum
<code>create_task</code> vs <code>gather</code>	task = fire-and-forget; <code>gather</code> = wait-for-all
async for/with	use <code>__aiter__</code> / <code>__anext__</code> and <code>__aenter__</code> / <code>__aexit__</code>
asyncio CPU block	<code>run_in_executor</code> to offload blocking/CPU work